

Rules as Files: Governing the Behaviour of LLM Agents Across Sessions, Machines and People

An Experience Report from a Single Production Operation

Anton Dzyatkovskii 

Platinum Software Development Company, Brisbane, Australia

dzyatkovskiy.a2@gmail.com · ORCID 0000-0001-7408-3054

<https://github.com/tonydz> · <https://tonydz.github.io>

September 2026

Abstract

An operating instruction given to an LLM agent survives one session. Written into the agent’s always-loaded configuration file it survives longer, until that file grows past the point where the model reliably attends to all of it, and then it decays silently: nothing errors, the rule simply stops firing. The same instruction given to a second machine, a second agent or a human assistant is a second copy, and copies drift. This experience report describes the governance layer we run in production to hold one behavioural codex across sessions, machines and people, published sanitized as an open repository: rules as one file each with typed frontmatter, precedence by recency and by author, a routing tree that gives each rule exactly one home, an always-loaded index carrying only trigger, essence and pointer, a journal of declined decisions, an intake ritual whose acceptance test is whether a parallel session that never saw the originating conversation would pick the rule up unaided, a quota requiring every rule to name the caller that will invoke it, a separation between writing a rule and compacting the index, and a protocol that turns an agent’s refusal into a numbered objection list the principal is invited to rebut. Two of the nine mechanics exist only because the design failed on us: one of those failures we audited and counted, finding that 19 of the 25 most recently added rules had no caller anywhere in the system and that one had sat unused for 42 days; the other we can only report, not quantify. We report which parts are not novel (most of them: this is policy-as-code, GitOps and the RFC process applied to natural-language instruction), what is grounded in a public artifact versus what remains our word, which results are self-judged, and what would falsify the approach. There is no controlled evaluation here: the numbers we report measure our own violations of our own design.

1 Introduction

Anyone who runs an LLM coding agent for more than a week will recognize this failure. You tell the agent something that matters: never push to that branch, always stamp who wrote a commit, do not send anything outward without asking. It works. The session ends. The instruction is gone.

The standard fix is to write it into the agent’s always-loaded configuration: `CLAUDE.md`, `AGENTS.md`, a system prompt, a memory file. This works too, for a while, and then produces a second failure that is worse because it is silent. The file grows, and nothing throws: the rule is present, quoted, correctly worded, and not followed. Be careful about what the literature does and does not license here. Long-context evaluations show a model’s use of its input depends strongly on position, with material in the middle used least reliably [10], and instruction-following benchmarks show adherence to explicit, verifiable instructions is well short of perfect [11]. Neither measures a rule sitting in an always-loaded configuration file, and neither establishes that adding rules degrades the ones already there: the extension to our case is an extrapolation, and it is ours, not theirs. What we can report is the operator’s experience of the outcome: watching only behaviour, you cannot distinguish “the rule is gone” from “the rule is there and was ignored”, and both feel like the model getting worse.

Add a second machine, and a third failure appears: the instruction now exists twice, and the two copies begin to diverge on the day one of them is edited. Add a human assistant, and a fourth: the person and the agent follow different documents, so the same situation produces two different behaviours depending on who handled it.

None of these are model problems. They are configuration-management problems, and configuration management has known answers: one source of truth, version control, explicit precedence, review, and a record of what was decided against. What is new is the material. The thing being versioned is natural-language instruction that a probabilistic reader interprets, that a human colleague also has to obey, and whose enforcement mechanism is, in the worst case, the model remembering to look.

This paper describes the layer we built for that and have run daily since July 2026 across a small fleet of machines and a mixed human/agent team, published in sanitized form as an open repository [1]. We call it a Bible functionally rather than reverentially: a versioned codex every actor loads before acting, with precedence rules and an overturn ritual (the repository’s `docs/why-a-bible.md` argues the word). The private content, which governs outreach, people and household matters, stays private; the skeleton is the contribution, and adopting it is an editing job rather than a project. The report is deliberately weighted toward where this design failed on us, since those are the parts a reader cannot reconstruct from a specification: two mechanics exist only as corrections to audited failures, one documented rule proved unkeepable, and a fact stated in two files of our own repository has drifted apart and is still drifting as we write.

Contributions. (i) **Nine mechanics** for rules-as-files governance of mixed human/agent teams, each stated so it can be copied: rule anatomy, precedence, routing, index-versus-body, the declined journal, the intake ritual, the door quota, the size-gate separation, and objection sparring (§3). (ii) The **two mechanics that exist only because we failed at them**, with the audit that produced them: 19 of 25 fresh rules unreachable, one idle for 42 days (§4). (iii) **Objection sparring**, a protocol that makes an agent’s “no” rebuttable, with the origin case in which, by the participants’ own judgment, 13 of 14 objections dissolved under a reframe the agent could not have generated (§5). (iv) A **grounding table** separating what a reader can verify in the public repository from what is unaudited operator experience (§6), and (v) a **novelty ledger** naming, per mechanic, the prior art that already covers it (§7).

What this paper is not. It is not an evaluation: no controlled comparison, no measurement that any mechanic reduces agent error, no external adoption data worth reporting (§8). The claim we defend is narrower: these mechanics make a rule’s location, currency, authority and reachability into facts that either exist or do not, instead of intentions.

2 The setting: what actually drifts

Three axes of drift, in the order they bite.

Across sessions. The unit of agent memory is the session; the unit of an operating rule is the operation. Written prose in an always-loaded file is the usual bridge across that boundary, and the file grows monotonically, because every new rule is an addition and nothing forces a deletion. Agent memory architectures address the retrieval half of this [13, 14], but retrieval is orthogonal to authority: the question is not only whether the agent can find the rule, but which of two conflicting rules governs.

Across machines and agents. A fleet is a distributed system whose configuration is prose. The moment the same rule exists in two files, they are two rules, and the second edit forks them. Our own public repository contains a live instance of this failure, which we leave in place because it is the honest illustration: the `README` and `FOR-ROBOTS.md` describe the operation as five machines, while `docs/why-a-bible.md` describes it as six.¹ The mechanism that prevents this is not discipline, it is having exactly one home per fact and pointers everywhere else.

Across people. The operation is human plus agent. A rule about how to answer a message has to be

¹Both files are in [1] today, checkable in under a minute. The fleet grew, one file was updated, the others were not, and no mechanism noticed: precisely the duplicate-homes failure M3 (§3) exists to prevent. The routing tree forbids this class for rules; we never applied the same discipline to descriptive facts about the operation, which is a gap in our practice rather than in the design.

#	Mechanic	What it is	Failure it prevents
M1	One rule = one file	Markdown file, typed frontmatter: <code>title</code> , <code>type</code> , <code>date_established</code> , <code>status</code> , <code>origin</code> , <code>authored_by</code> , <code>audience</code> , optional <code>supersedes</code> .	A rule that cannot be cited, dated, superseded or attributed, because it is a paragraph inside another rule.
M2	Precedence	Newer beats older on the same topic; <code>origin: owner</code> beats agent-inferred regardless of date; explicit <code>supersedes</code> beats both. Superseded files are kept and linked forward.	Two live rules on one topic and no way to say which governs. Deleting history to resolve it.
M3	Routing tree	Four homes, asked in order: could a human execute it (codex), is it agent-internal (agent config), is it a fact not a behaviour (memory), is it deterministic “every time X” (a hook). One home, pointers elsewhere.	Duplicate homes that drift. Deterministic automation written as prose a model must remember to apply.
M4	Index vs body	The always-loaded layer carries only trigger, essence, pointer. The body loads on demand.	Context bloat and the silent non-following that comes with it [10, 11].
M5	Declined journal	Append-only log of what was decided against, with the reason and an explicit <code>Revisit if</code> condition. Scanned before proposing anything structural.	The rejected idea returning three weeks later as a fresh proposal, because memory stores only what is true now.
M6	Intake ritual	A rule spoken in conversation is routed the same sitting: written, linked, indexed, reported. Test: would a parallel session that never saw the conversation pick it up unaided?	Rules that die in scrollback while everyone believes they were captured.
M7	One rule = one door	Every rule names, in the same sitting, the caller that invokes it: a skill, a command, a scheduled job, a hook. The codex is not a door: it is read, not called.	The filed, linked, indexed rule that nothing ever calls (§4).
M8	Writing never gated by size	The session that hears a rule writes it, over threshold or not, and reports the size in one line. Compacting the index is a separate daily job with its own owner.	“The file is too big” becoming the mechanism by which rules are lost.
M9	Objection sparring	A no-go verdict is a numbered objection list plus an invitation to rebut; what survives becomes a boundary inside the decision, not a veto outside it (§5).	An agent’s refusal ending a debate the principal’s context would have won.

Table 1: The nine mechanics. M1–M5 and M8 are file-level properties, present or absent on inspection; M6, M7 and M9 are obligations on a process, and M7’s presence is checked by finding the caller, not by reading the rule.

executable by a person and by a model, or the two produce different behaviour on the same input. Hence a design property a purely machine-facing config does not need: rules carry an `audience` field, and those marked both are written in language a new human assistant can follow on their first day.

3 The architecture: nine mechanics

The full normative text is the public specification [1]; Table 1 is the summary, and the paragraphs after it give the reasoning that the specification does not carry.

M1–M2: authority is metadata, not tone. The interesting field is `origin`. A rule stated by the principal and a rule the agent inferred from three observations look identical once written, and must not be overturned on the same terms: the second may be revised by any session with better information, the first only by its author. Encoding that as a field rather than a convention is what lets an unattended session decide, at three in the morning, whether it may rewrite something. The other field that earns its place is `date_established`, because the dominant conflict in a living codex is not contradiction but staleness. It is also the weakest link in the design, and §8 says why.

Supersede-instead-of-edit costs the most discipline and is the property we would defend hardest: a superseded rule stays on disk with `status: superseded` and a forward link. In a fleet, some session, cache or human is still holding the old version, and their only way to discover they are wrong is to find the old file and read the pointer. A deleted rule cannot correct anybody.

M3–M4: the fourth branch, and what the index is. The four homes are logical, not directories. Three of them are ordinary information architecture; the fourth is a policy statement about what prose is for. If a rule

has the shape “every time X happens, do Y” and X is machine-detectable, it does not belong in a codex at all: written as prose it fires when a model remembers it, written as a hook it fires. Putting that question inside the routing tree means it gets asked at intake, when moving the rule is still cheap. The always-loaded line, in turn, is a trigger rather than a compressed rule: enough for the agent to recognize that this situation is governed and to know where the text is, and a line that tries to be self-sufficient defeats the mechanic. What the split buys is a smaller always-loaded surface; what it costs is a new dependency on the agent actually following the pointer, which we do not measure (§8).

M5–M6: the journal, and what counts as captured. Memory stores what is true. Nothing in a standard agent memory records that something was considered and rejected, so the agent cannot distinguish an idea nobody has had from one the principal already refused; the cost is not the wasted proposal but the principal’s trust that the agent remembers anything. The entry carries a `Revisit if` condition, turning “no” into “no, until”. The intake ritual’s value lies entirely in its acceptance criterion: writing a rule down feels like capturing it, while the test asks whether a session with no access to the originating conversation would apply it. That test forces the index line, the links and the door.

4 The two mechanics we only have because we failed

M7 and M8 are not design; they are scar tissue, and both were found by audit rather than by reasoning.

The door quota. In August 2026 we audited the 25 most recently added rules of our own codex against a simple question: what, anywhere in the system, actually invokes this? The answer, published in the specification [1]: **19 of 25, or 76%, had no caller at all.** They were correctly written, correctly filed, correctly linked and indexed, and fired only when a human happened to remember them. One, a rule requiring root-cause analysis before building any new mechanism, had sat **42 days** without a single application. This is one snapshot of one window, taken by the people who wrote the rules; treat the figure as the reason the mechanic exists, not as a rate.

The correction is a quota: a new rule names its door in the same sitting, cheapest form first, where a line inside a procedure that already runs beats standing up a new job. Two failure shapes deserve separate names, because the second is the dangerous one:

- **No door.** Nothing calls it. Detectable by grep, embarrassing, easy to fix.
- **Fictitious door.** The rule’s name appears inside a procedure as prose, with no command beside it. A search for “is this wired up?” returns a confident yes. It executes nothing. This shape survives audits that the first shape does not, because it defeats the audit’s own instrument.

The quota binds the agent, not the principal: the principal may state any rule at any time, and the agent owes the door. Where no door is possible today, the obligation is to record that in a short list of doorless rules with reasons, so the gap is a visible number. This is M6’s acceptance test applied one layer out: convert a belief (“we have that rule”) into an artifact (“here is the line that calls it”).

The size gate. M4 keeps the always-loaded index small, and a small file that grows produces a predictable temptation: this index is at its limit, I will add the rule later. We watched that reasoning consume rules that had gone through the entire intake ritual. The correction splits two jobs that look like one and gives them different owners. Accepting a rule is done by whichever session heard it, unconditionally, with a one-line size report attached; compacting the index is a separate, scheduled, single-owner job, never done inline on somebody else’s task and somebody else’s judgment.

It would be neater to claim this does not conflict with M4, and it does. Between the moment the index crosses its threshold and the next compaction, M8 overrides M4: rules are written into a file already too large, which is the exact condition M4 exists to avoid, and in that window a new rule can be as unreliable as one buried in the middle of a long prompt. We accept the trade because the two failures are not symmetric. The M4 failure is temporary and self-correcting, since the scheduled compaction ends it; the M8 failure, a rule refused at intake because a file looked full, is permanent, and nothing later goes looking for it. A size check in front of an intake is

therefore a measurement for the report, never a brake. The honest statement of the mechanic is not “no conflict” but “M8 wins, and here is what it costs”.

The general form of both lessons: a mechanic that depends on an actor’s judgment in the moment will be defeated by whatever that actor is optimizing for in the moment. Give the obligation an owner and an artifact.

5 Verdicts as debate: objection sparring

The mechanic with the largest effect on how the operation actually feels is M9, and it exists because of a single argument we recorded and then published [2].

Asked whether to package and ship a version of the system, the agent returned a no-go backed by fourteen numbered objections: a saturated market of mature free alternatives, single-user architecture, legal exposure in acting on someone’s behalf, retention that collapses inside a month, support load beyond a solo operator, incumbents moving into the niche. Under the rule, that verdict was delivered as the list with an invitation to rebut, rather than as a sentence.

The principal’s rebuttal did not attack the objections. It changed the question. The agent had been answering “should we build a paid product for strangers”; the intent had been “should we give the blueprints away and teach”. Under the second question, 13 of the 14 objections dissolved in one pass: retention does not matter for something nobody subscribes to, support is provided by each adopter’s own agent, single-user architecture is the design goal rather than a limitation. The fourteenth stood and became a written boundary inside the resulting decision.

How that count should be read. “Dissolved” was judged by the two participants in the argument, who also authored the rule and this paper. There was no blind rater, no pre-registered criterion, no independent replay. The number records what the debate concluded, not the quality of the arguments; what is checkable is the artifact trail, since the rule and the narrative are both public [1, 2].

Three properties make this a mechanic rather than a manner:

1. **The list is a questionnaire.** A refusal delivered as prose buries its own premises. Enumerated, each objection becomes separately answerable, and the principal’s answers are data the agent could not have generated, because the missing information was the frame.
2. **Dissolution must be argued, non-dissolution must be said.** An objection a rebuttal did not close is reported as standing. This is the anti-sycophancy clause and the part needing enforcement, since agreement is the path of least resistance for a model here [12].
3. **An overturned verdict is a real decision.** It produces a memo, a *supersedes* pointer, and an OVERTURN mark in the declined journal (M5), so no parallel session cites the dead “no” as live.

The protocol is symmetric: when the principal rejects the agent’s proposal, the agent may ask for objections in the same form and work them, subject to the declined journal’s rule that a re-pitch requires new information. It does not weaken hard gates, since money, irreversible actions, outbound publication and secrets require the principal’s approval regardless of who won, and those gates are enforced by code rather than by the codex [3]. Winning a debate does not grant authority; it changes a decision.

6 Grounding: artifact, experience, opinion

Experience papers fail when the reader cannot tell which sentences are checkable. Table 2 sorts every substantive claim in this paper into three tiers.

One Tier A row deserves its own sentence, being the most useful thing we learned about writing rules at all. The repository once carried a rule that releases would be cut twice a week on fixed days; between 4 July and 4 August 2026 it was not kept once, and the README now says so. The rule was tied to the calendar rather than to the work. Its replacement, that every noticeable change ships as a release, is keepable because its trigger is an event that actually occurs. A rule triggered by a date fires whether or not there is anything to do, and one that

Tier A: public artifact	Tier B: operator instrumentation, unaudited	Tier C: our design opinion
The nine mechanics, the frontmatter schema, precedence and routing tree, verbatim in [1].	The scale of the private codex: a vault the repository describes as 128k notes, five to six machines, a mixed human/agent team. Our own counting, not auditable.	That four homes is the right number, and a fifth needs an argument.
The door audit: 19 of 25 rules doorless, one idle 42 days, in [1] §9.	How often precedence actually fires in daily operation. We do not instrument it and will not claim it.	That M7 (doors) is the highest-leverage of the nine, ahead of the ones that look more architectural.
The objection-sparring case, 14 objections, 13 dissolved [1, 2]. The count is self-judged (§5); the artifacts are not.	The daily intake load, and whether the mechanics reduced our own error rate. We believe the latter and cannot show it.	That supersede-instead-of-edit earns its discipline cost in a fleet, and would not on one machine.
Repository history: first commit 2 July 2026, 33 commits, 7 releases (v0.1.0 to v0.1.6, latest 5 September 2026), MIT, via the GitHub API. The five-versus-six machine discrepancy in our own repository (§2). The cadence rule we could not keep, admitted in the README.		That prose is the wrong medium for deterministic “every time X” rules, even when the model follows prose well.

Table 2: Grounding. Tier B is our word; we mark it rather than dressing it as measurement.

regularly fires with nothing to do teaches everyone to ignore it: a decay path independent of context windows, against which no file hygiene protects.

7 Related work, and where we claim nothing

This design is an assembly of established practice applied to a new material. The honest presentation is a ledger (Table 3) that says “no novelty” wherever that is the verdict, which is most rows.

Four neighbours deserve their own paragraph.

The agent config file itself (AGENTS.md, CLAUDE.md and relatives [19]) is what nearly every reader already uses, and the honest comparison is not favourable in the way a paper usually wants: that convention already gives one versioned, reviewable, repository-local instruction file, which is most of the value. Everything here is what we had to add once one file stopped being enough: a precedence field, so two rules on one topic resolve without a human; an audience field, so a human assistant and an agent read the same rule; a body/index split; a declined journal, which the convention has no place for; and the door quota, which it does not raise at all. An adopter with one machine, one agent and one operator should use the convention and stop reading here. These mechanics start paying at the second machine and the second human.

Constitutional AI [7] governs a model’s behaviour with a written document of principles applied during training and self-critique, and the instruction hierarchy [8] trains a model to rank its instruction sources. Both are the obvious relatives and both operate one level down, in the weights. Everything here operates at the level of an *operation*, governs humans as well as agents, is edited daily by its subjects, and has no training-time component. A constitution in our sense is a coordination artifact, not an alignment technique, and must never be mistaken for an authorization mechanism: a rule in a file does not stop an agent persuaded to ignore it, which is why irreversible classes are gated by code [3].

Policy-as-code [15] is the closest engineering analogue and the sharpest contrast: a policy engine evaluates deterministically, and nothing in this paper does. When a rule can be made deterministic our own routing tree says it must leave the codex and become a hook, which is an admission that policy-as-code is strictly better wherever it applies. The codex is what remains for rules requiring judgment, and for rules whose executor is sometimes a person.

Agent memory [13, 14] solves retrieval and forgetting; this paper is about authority and reachability, and the two compose. Companion work covers neighbouring layers of the same operation: control-plane failure modes [4], delegated authority [3], the fleet experience report [5], and whether operator instructions survive context compaction [4], the mechanism-level version of the decay managed here at the document level.

Mechanic	Prior art that already covers it	Our delta
M1 one rule = one file	Policy-as-code: policies as versioned source, evaluated by an engine rather than remembered [15]. Architecture decision records, one file per decision [17]. Controlled-document practice in quality systems: identity, owner, revision, effective date.	No novelty in structure. The material differs: the “engine” is a probabilistic reader, so the file must also be human-executable.
M2 precedence	The RFC process: numbered, dated documents that obsolete and update one another and are never deleted [18]. Version identifiers and changelog practice. At the model level, training an explicit instruction hierarchy over privileged and unprivileged sources [8].	No novelty , and [8] solves the harder version of it in the weights. Ours is the document-level analogue for a mixed human/agent team: one field, origin , overriding recency.
M3 routing tree	Infrastructure as code and GitOps: declared desired state in version control, reconciled by machine rather than by memory [16].	No novelty in principle. The fourth branch, deterministic rules leaving prose for hooks, is the specific application to agent operations.
M4 index vs body	Retrieval-augmented and paged agent memory [13, 14]; long-context attention limits [10]; instruction-following degradation [11].	No novelty : this is paging, decided at authoring time instead of at retrieval time.
M5 declined journal	Decision records that capture rejected options [17]; the “alternatives considered” section of a design doc.	Emphasis, not novelty . Making it a standing journal scanned <i>before</i> proposing, with an explicit revisit condition, is the operational half those formats leave to discipline.
M6 intake ritual	Docs-as-code review workflows; conventions such as AGENTS.md for instructing coding agents [19].	Ours, minor : the acceptance test (would an uninvolved parallel session apply this?) as the definition of captured.
M7 one rule = one door	Requirements traceability, where every requirement must link to the artifact that implements and the test that exercises it [9]; dead-code analysis; the principle that unexercised configuration is unreliable.	Weaker than we first thought : a door is a traceability link, and [9] states the obligation generally. Ours is the naming of the two failure shapes (absent versus fictitious door) and a count of our own violations.
M8 size gate	Separation of concerns; single-writer ownership of a file.	Ours, minor : naming the anti-pattern (a size check functioning as a brake on intake) and splitting the two jobs by owner.
M9 objection sparring	Devil’s advocacy and dialectical inquiry in decision research; sycophancy as a known failure of aligned models [12].	Ours, in this form : the numbered-list contract, undissolved objections becoming boundaries not vetoes, and the OVERTURN record tying the outcome into precedence.

Table 3: Novelty ledger. Six of nine mechanics contribute nothing new in principle; the contribution, if any, is assembly, minimality and the two audited failures.

8 Limitations, and what would falsify us

This is $n = 1$, with no evaluation. One operation, one principal, one primary agent product, a household-scale fleet, and every mechanic confounded with the fact that its authors wrote the rules and wanted them to work. We did not run governed against ungoverned, and cannot say the codex reduced error; only that specific gaps became visible and countable once the mechanics existed. The two numbers we report (76% doorless, 42 idle days) measure our own violations of our own design: evidence that the audit works, and none at all that the design does. The evaluation we would want is within reach of anyone with two operations or one operation and patience: hold the rule set fixed, vary the index/body split, and measure the rate at which a governed situation is handled according to its rule, blind-scored by someone who did not write it. We have not run it.

Precedence has no clock we trust. `date_established` is a field a human or an agent writes, not a logical clock. Two sessions on two machines can author conflicting rules in the same minute, a rule can be dated by the day it was spoken rather than filed, and the ordering of the underlying commits need not agree with either. We resolve collisions by asking the principal and have no automatic semantics for concurrent authorship: at fleet scale this is the first thing we expect to break, and a design that wanted it solved should reach for causal ordering rather than dates.

The index/body split moves a risk rather than removing one. A rule whose index line was recognized but whose body was never loaded fails in exactly the silent way the split was meant to prevent, and we would not currently detect it.

One central number is self-judged. The 13-of-14 dissolution in §5 was scored by the participants, with no blind rater and no pre-registered criterion.

Adoption is small, and we will not inflate it. As of 10 September 2026 the repository has 9 stars, 1 fork,

33 commits and 7 releases, and we know of no external reproduction. Anyone reporting that they adopted the mechanics and that one failed would be more useful to us than any current signal.

What would falsify the approach. (i) An operation adopting the mechanics and finding the index still decays: the split is not the operative variable. (ii) A doorless rule that fires reliably anyway: recall suffices and the quota is ceremony. (iii) Objection sparring producing agreement at the same rate as unstructured refusal: the format is politeness. (iv) A fleet where supersede-instead-of-edit causes more stale-file confusion than it corrects, the failure we expect first at larger scale. We would report any of these; the declined journal exists so our own reversals stay on the record.

Code, data and contact

The specification, templates and sanitized examples are a live, open repository: <https://github.com/tonydzi/claude-bible> (MIT), docs site <https://tonydzi.github.io/claude-bible/>. Everything cited here as Tier A is readable there today, including the door audit in docs/SPEC.md §9 and the objection-sparring rule in examples/. Related open components of the same operation, all under <https://github.com/tonydzi>: delegated-authority control model (agent-leash), out-of-band approval gate (agent-approval-gate), multi-machine consensus (claw-consensus), control-plane failure casebook (agent-control-plane-casebook), fleet deployment with landing proof (fleet-deploy), an independent review panel gating “done” (secondop-panel), and publication tooling that ships internals without leaking the operation (oss-publish).

Corrections are the contribution we value most, and a failed reproduction is worth more to us than a star: open an issue on claude-bible. Author: Anton Dziahtkovskii, dzyatkovskiy.a2@gmail.com · ORCID 0000-0001-7408-3054 · <https://github.com/tonydzi> · <https://tonydzi.github.io>. Companion preprints: [5, 4, 6].

Authorship and responsibility. This paper and the accompanying repositories are drafted by an AI agent operating as the author’s synthetic co-founder, under the author’s editorial control; the author is the responsible human for every claim made here, every commit carries provenance trailers naming tool, model, machine and operator, and corrections are logged publicly. The repository described here is maintained under the rules it describes, which is a demonstration that they can be lived with, not a demonstration that they work.

References

- [1] A. Dziahtkovskii. claude-bible: a governance skeleton for one behavioral codex across a principal, their agents and their team. <https://github.com/tonydzi/claude-bible>, 2026. MIT. Specification: docs/SPEC.md.
- [2] A. Dziahtkovskii. The day my AI cofounder told me “no”. docs/the-day-my-ai-said-no.md in [1], 2026.
- [3] A. Dziahtkovskii. Keeping agents on a leash: an eight-domain control model for agents with delegated authority. Preprint and repository, <https://github.com/tonydzi/agent-leash>, 2026; with the approval gate it references, <https://github.com/tonydzi/agent-approval-gate> (MIT).
- [4] A. Dziahtkovskii. Silent by construction: control-plane failures in a production LLM agent fleet. Preprint and repository, <https://github.com/tonydzi/agent-control-plane-casebook>, 2026; and Instructions that never arrive: whether operator compaction instructions reach the summarizer. Preprint, 2026.
- [5] A. Dziahtkovskii. Operating a human-governed multi-machine LLM agent fleet: an experience report. Preprint, 2026. doi:10.5281/zenodo.22639714.
- [6] A. Dziahtkovskii. Companion preprints, 2026: homeostatic governance, doi:10.5281/zenodo.22639712; Le Chatelier tokenomics, doi:10.5281/zenodo.22639710; graph expansion in personal-corpus retrieval, doi:10.5281/zenodo.22639718.
- [7] Y. Bai et al. Constitutional AI: harmlessness from AI feedback. arXiv:2212.08073, 2022.
- [8] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, A. Beutel. The instruction hierarchy: training LLMs to prioritize privileged instructions. arXiv:2404.13208, 2024.
- [9] O. Gotel and A. Finkelstein. An analysis of the requirements traceability problem. In *Proc. IEEE Int. Conf. on Requirements Engineering*, 1994.
- [10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: how language models use long contexts. *TACL*, 12:157–173, 2024. arXiv:2307.03172.
- [11] J. Zhou et al. Instruction-following evaluation for large language models. arXiv:2311.07911, 2023.
- [12] M. Sharma et al. Towards understanding sycophancy in language models. arXiv:2310.13548, 2023.
- [13] C. Packer et al. MemGPT: towards LLMs as operating systems. arXiv:2310.08560, 2023.
- [14] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein. Generative agents: interactive simulacra of human behavior. *UIST*, 2023. arXiv:2304.03442.
- [15] Open Policy Agent and the Rego policy language. Cloud Native Computing Foundation. <https://www.openpolicyagent.org>.
- [16] OpenGitOps: principles of declarative, version-controlled, continuously reconciled operations, v1.0, CNCF App Delivery TAG, 2021–2026, <https://opengitops.dev>; and K. Morris, *Infrastructure as Code*, O’Reilly, 2nd ed., 2020.
- [17] M. Nygard. Documenting architecture decisions. Cognitect blog, 15 November 2011.
- [18] S. Bradner. The Internet standards process, revision 3. RFC 2026, IETF, 1996.
- [19] AGENTS.md: an open convention for instructing coding agents, 2025–2026. <https://agents.md>.